

TP1 – Diagramas de Estado – Modelado

Objetivos

Aproximación a **3.1.- TA134 - TdSE - 1er Proyecto**

Describir la solución completa **Intelligent Parking Management System**

Describir la implementación de **Parking Ticket Dispenser Machine (Entry)**

Enunciar y describir **Eventos** y **Acciones** de cada modelo de **Parking Ticket Dispenser Machine (Entry)**

Enunciar y describir **Estados y Excitaciones** de cada modelo de **Parking Ticket Dispenser Machine (Entry)**

Usar **Itemis Create Cloud Editor** e **Itemis Create Desktop**

Generar proyecto **Itemis Create**

Editar/simular/depurar **diagramas de estado** (modelos)

Usar **Git** y **GitHub**

Generar/actualizar **repositorio**

Generar/actualizar **archivos** del tipo **README.md** (**Markdown**)

Diagramas de Estado – Modelado

My First Statechart (Modeling)

Sensor Statechart (Modeling)

System Statechart (Modeling)

Actuator Statechart (Modeling)

Referencias

3.1.- TA134 - TdSE - 1er Proyecto (clase teórica 3, presentación 1)

3.2.- TA134 - TdSE - Máquina de Estados (clase teórica 3, presentación 2)

Actividades

1er encuentro

TP1-00 – Problem approach (algunas son **Tarea para el hogar**)

TP1-01 – My First Statechart (algunas son **Tarea para el hogar**)

TP1-02 – Sensor Statechart

2do encuentro

TP1-03 – System Statechart

TP1-04 – Actuator Statechart

Resolución en grupos de **3** alumnos, requiere **presentación** por parte de un **responsable** del grupo

TP1 – Actividad 00 – Problem approach

Objetivos

Aproximación a [3.1.- TA134 - TdSE - 1er Proyecto](#)

Describir la solución completa **Intelligent Parking Management System**

Describir la implementación de **Parking Ticket Dispenser Machine (Entry)**

Enunciar y describir **Eventos** y **Acciones** de cada modelo de **Parking Ticket Dispenser Machine (Entry)**

Enunciar y describir **Estados y Excitaciones** de cada modelo de **Parking Ticket Dispenser Machine (Entry)**

Usar **Git** y **GitHub**

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** ([Markdown](#))

Paso 01: Crear un nuevo [repositorio](#) en [GitHub](#) para almacenar su **TP1**, pasos:

[GitHub](#): **Repositories => New =>**

Repository name: **tdse-tp1_**[Año-Cuatrimestre](#) **_Curso-Grupo**

Description: **# FIUBA - Electrónica - Taller de Sistemas Embebidos - Trabajo Práctico N°: 1 - Diagramas de Estado - Modelado**

Initialize this repository with:

Tildar **Add a README file**

=> Create repository

Año-Cuatrimestre es: **2026-1erC** o **2026-2doC** ... etc. (el correspondiente al Año y Cuatrimestre de cursada)
Curso-Grupo es: **1-01** o **1-02** ... etc. (el correspondiente al Curso y Grupo de cursada)

Paso 02: Invitar al **repositorio**, como [colaboradores](#) a los [docentes auxiliares](#) de su curso, responsables de la aprobación de su **TP1**, pasos:

[GitHub](#): **Repositories => Settings => Collaborators => Tildar: Add people =>**

en **Find people** => tipear: **Docente_Auxiliares-FIUBA**

Tildar: **Add Docente_Auxiliares-FIUBA**

Una vez que los docentes auxiliares de su curso, **hayan aceptado** la invitación a ser [colaboradores](#), podrá agregarlos como [Revisores](#)

PasPaso 03: Asentar los detalles de las **entregas** en archivos del tipo **README.md** ([Markdown](#), no docx o pdf u otros formatos), **editar** el archivo **README.md** y **modificar** su contenido, pasos:

GitHub: [README.me](#) => **Edit this file** => **borrar** su contenido => **Copiar** y **Pegar** lo siguiente:

```
# FIUBA - Electrónica - Taller de Sistemas Embebidos
## Trabajo Práctico N°: 1 - Diagramas de Estado - Modelado
### Año-Cuatrimestre - Curso-Grupo

### Responsable de la entrega:
| Padrón | Apellidos, Nombres | Fecha | Deadline |
| :----- | :----- | :----- | :----- |
| XXXXXX | YYYY, ZZZ | | Semana 04 |
```

Actualizar el archivo **README.md** con los datos del responsable de la entrega (**un alumno diferente para cada TP**):

Año-Cuatrimestre, **Curso-Grupo**, **Padrón**, **Apellidos** y **Nombres**

Paso 04: Completar [TA134 - TdSE - TP1 - Registro de Entrega](#).

Paso 05: **Asentar** los detalles de las **entregas** en archivos del tipo **README.md** ([Markdown](#), no docx o pdf u otros formatos), **crear/editar** el archivo del tipo **README.md** y **modificar** su contenido, pasos:

Crear el archivo **tdse-tp1_00-problem_approach.md**, **editar** y **modificar** su contenido:

Describir la solución de **COMA Electronics**, [referencia](#) de [3.1.- TA134 - TdSE - 1er Proyecto](#), Intelligent Parking Management System - Automated Parking System - Parking Ticket Dispenser Machine (**Entry**).

Describir la **implementación** de **Parking Ticket Dispenser Machine (Entry)** de [3.1.- TA134 - TdSE - 1er Proyecto](#).

Enunciar los **modelos** para **describir** el **comportamiento** de c/u de los **módulos** de código **C** del tipo temporizado (Update by Time Code, period = **1mS**), para **escrutar** => **Sensor**, **procesar** => **System**, **actuar** => **Actuator**.

De ser posible:

Reemplazar por **pulsadores**, **interruptores dip switch** (Digital Inputs) a los **sensores** reales (Camera, Button y Sensor Coil)

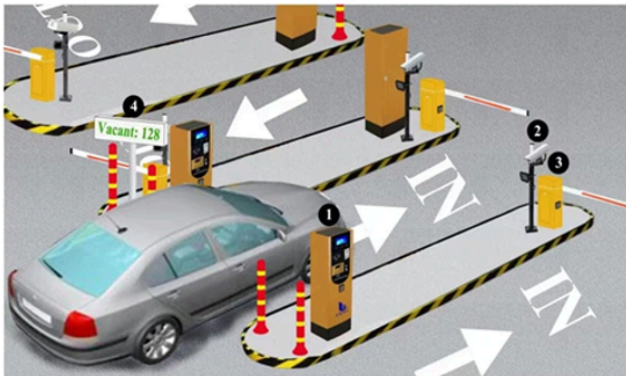
Reemplazar por **leds** (Digital Outputs) a los **actuadores** reales (Display, Printer, Barrier y Sever)



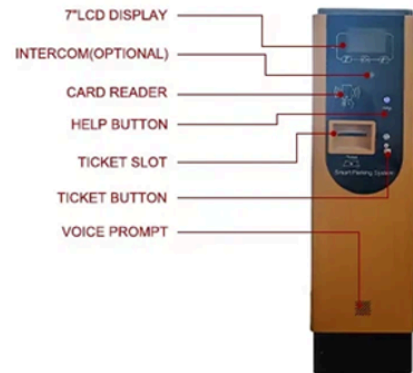
Parking Ticket Dispenser Machine (Entry)

<https://www.comaparking.com/parking-management-system/parking-ticket-system/parking-ticket-dispenser-machine.html>

COMA PARKING ACCESS CONTROL SYSTEM



1. TICKET MACHINE with video INTERCOM and radar trigger
2. Motorize camera with auto light
3. High speed barrier gate with radar trigger
4. Vacant number LED display



17

Tarea para el hogar

Paso 06: Asentar los detalles de las entregas en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear/editar** el archivo del tipo **README.md** y **modificar** su contenido, pasos:

Crear el archivo **tdse-tp1_00-sensor.md**, editar y modificar su contenido:

Enunciar y describir los **Eventos** y **Acciones** del **modelo Sensor** (un solo botón) necesarios para describir el **comportamiento** del **módulo** de **código C** del tipo temporizado (Update by Time Code, period = 1mS) para "escrutar", necesario para la **implementación** referida en el **TP1 - Actividad 00 - Paso 05**.

Un **Sensor** (un solo botón) del tipo **binario** genera **2** (dos) **Eventos** (reflejan el valor **binario**) asociados a su **Posición** y al menos **2** (dos) **Acciones** (reflejan que hubo un cambio de **Posición** del botón)

Convención de identificadores

event => EV_BTN_NAME
signal => EV_SYS_NAME

state => ST_BTN_NAME
timer => tick // 0, DEL_BTN_NAME

Las **Posiciones** del botón (not pressed/pressed) son trigger (**Eventos**) para el **modelo Sensor**.

Las **Acciones** del **modelo Sensor** pueden ser **signals** (**Eventos**) para el **modelo System** o bien ejecutar **funciones** o bien "inicialización/modificación" de **variables** de control (**timer**); variables que pueden usarse como **guard** para condicionar transiciones (**trigger** [guard] / effect).

Un **pulsador**, también conocido como **botón**, es un dispositivo mecánico que activa o desactiva un circuito eléctrico al ser presionado o soltado (interrumpiendo o permitiendo el flujo de corriente eléctrica en un circuito). Un **interruptor dip switch** se comporta de manera similar a un **pulsador**, pero no es necesario mantenerlo presionado.

El rebote de un **pulsador**, también conocido como "**switch bounce**" o "**debouncing**", es un fenómeno mecánico que ocurre cuando un pulsador, al ser presionado y liberado, no realiza una transición limpia y precisa entre los estados de

"abierto" y "cerrado". En lugar de un cambio instantáneo, los contactos internos del pulsador vibran momentáneamente, generando múltiples y rápidas fluctuaciones en la señal eléctrica antes de estabilizarse.

Es necesario recurrir a **timer (tick)**.

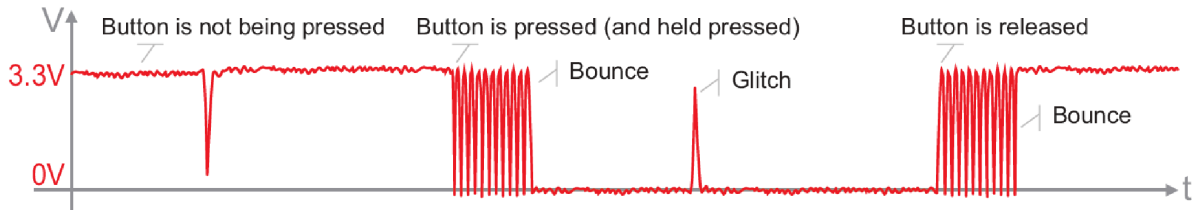


Figure 4.6 Voltage signal over time for a given button, including typical glitches and bounces.

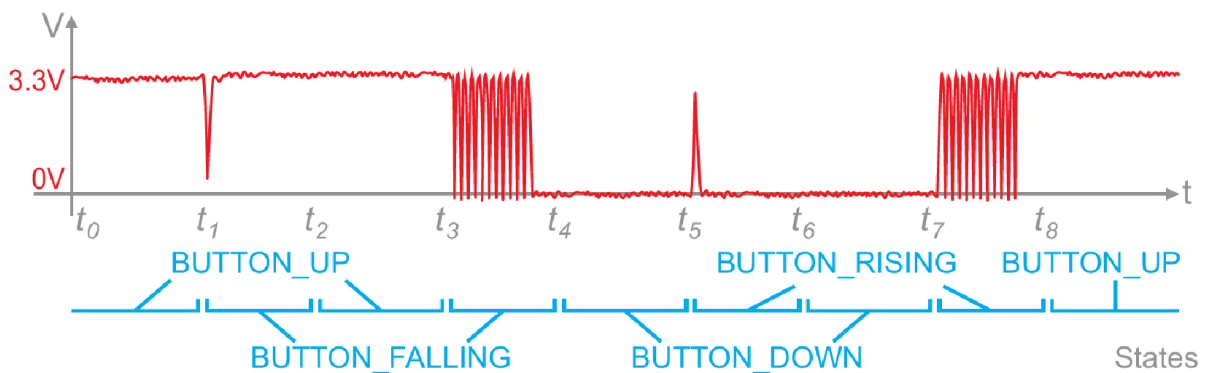


Figure 4.8 Voltage signal over time for a given button as it is pressed or released.

Paso 07: En el archivo **tdse-tp1_00-sensor.md**, **editar** y **modificar** su contenido:

Enunciar la **tabla de Estados y Excitaciones** del **modelo Sensor** (**un solo botón**) necesarios para **describir** el **comportamiento** del **módulo** de **código C** del tipo temporizado (Update by Time Code, period = 1mS) para **"escrutar"**, necesario para la **implementación** referida en el **TP1 – Actividad 00 – Paso 05**.

Sensor Statechart - State Transition Table				
Current State	Event	[Guard]	Next State	Actions

Paso 08: **Asentar** los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear/editar** el archivo del tipo **README.md** y **modificar** su contenido, pasos:

Crear el archivo **tdse-tp1_00-system.md**, **editar** y **modificar** su contenido:

Enunciar y **describir** los **Eventos** y **Acciones** del **modelo Sensor** (**un solo botón**) necesarios para **describir** el **comportamiento** del **módulo** de **código C** del tipo temporizado (Update by Time Code, period = 1mS) para **"escrutar"**, necesario para la **implementación** referida en el **TP1 - Actividad 00 - Paso 05**.

Las **Acciones** del **modelo System** pueden ser **signals (Eventos)** para el **modelo Actuator** o bien ejecutar **funciones** o bien "inicialización/modificación" de **variables** de control (**timer**); variables que pueden usarse como **guard** para condicionar transiciones (**trigger [guard] / effect**).

Paso 09: En el archivo **tdse-tp1_00-system.md**, editar y modificar su contenido:

Enunciar la tabla de **Estados y Excitaciones** del **modelo System** (un solo sistema) necesarios para **describir** el **comportamiento** del **módulo** de **código C** del tipo temporizado (Update by Time Code, period = **1mS**) para **“procesar”**, necesario para la **implementación** referida en el **TP1 – Actividad 00 – Paso 05**.

System Statechart - State Transition Table				
Current State	Event	[Guard]	Next State	Actions

Paso 10: Asentar los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear/editar** el archivo del tipo **README.md** y **modificar** su contenido, pasos:

Crear el archivo **tdse-tp1_00-actuator.md**, editar y modificar su contenido:

Enunciar y **describir** los **Eventos** y **Acciones** del **modelo Actuator** (un solo led) necesarios para **describir** el **comportamiento** del **módulo** de **código C** del tipo temporizado (Update by Time Code, period = **1mS**) para **“escrutar”**, necesario para la **implementación** referida en el **TP1 - Actividad 00 - Paso 05**.

Las **Acciones** del **modelo Actuator** pueden ser **signals** (**Eventos**) para el otros modelos o bien ejecutar **funciones** o bien “inicialización/modificación” de **variables** de control (**timer**); variables que pueden usarse como **guard** para condicionar transiciones (**trigger** [**guard**] / **effect**).

Un **led**, o diodo emisor de luz, es un dispositivo semiconductor que emite luz cuando se le aplica una corriente eléctrica.

Un **led** puede estar **encendido**, **apagado**, **titilando**, **un pulso**, **pulsos (N)**, etc., etc.

Es necesario recurrir a **timer** (**tick**).

Paso 11: En el archivo **tdse-tp1_00-actuator.md**, editar y modificar su contenido:

Enunciar la tabla de **Estados y Excitaciones** del **modelo Actuator** (un solo led) necesarios para **describir** el **comportamiento** del **módulo** de **código C** del tipo temporizado (Update by Time Code, period = **1mS**) para **“actuar”**, necesario para la **implementación** referida en el **TP1 – Actividad 00 – Paso 05**.

Actuator Statechart - State Transition Table				
Current State	Event	[Guard]	Next State	Actions

Paso 12: Almacenar los archivos de la actividad, en el **repositorio** creado en **GitHub**, en el **TP1 – Actividad 00 – Paso 01**.

TP1 – Actividad 01 – My First Statechart

Objetivos

Usar **Itemis Create Cloud Editor**

Generar proyecto **Itemis Create**

Editar/simular/depurar **diagrama de estado** (modelo): **My First Statechart** (Modeling)

Usar **Git** y **GitHub**

Generar/Actualizar repositorio por línea de **comando**

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Paso 01: Ingresar al sitio de **Itemis Create Cloud Editor** => <https://create.itemis.io/login>

Herramienta vía **navegador web**, funciona correctamente con la **versión actualizada** de **Chrome**.

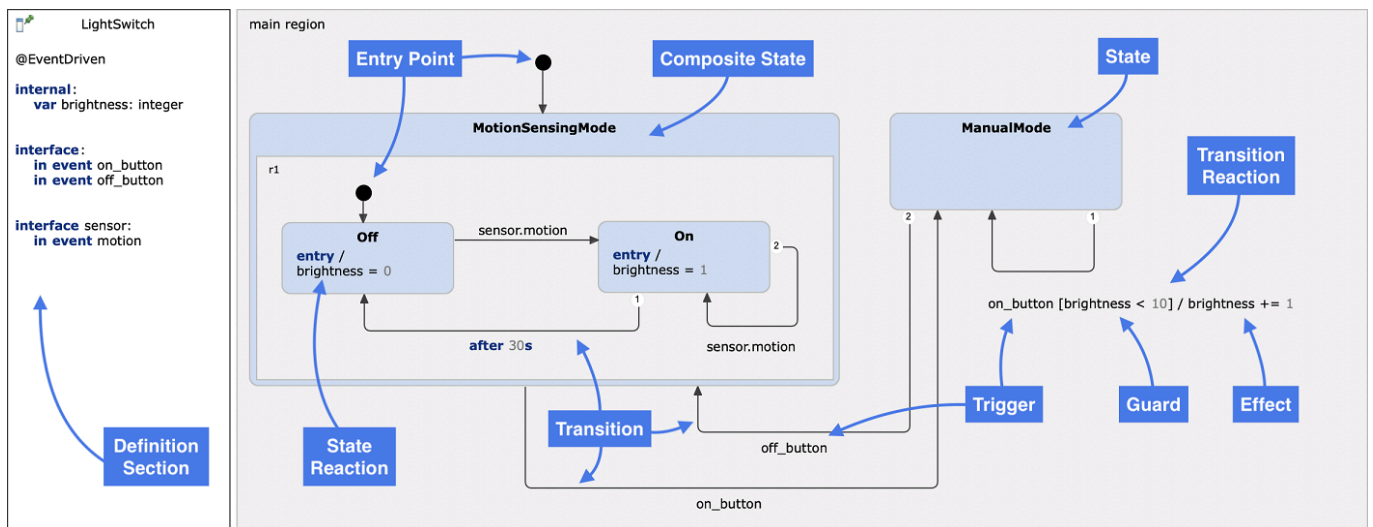
Paso 02: Ingresar por primera vez sitio de **Itemis Create** Cloud Editor => <https://create.itemis.io/login>, se genera automáticamente el **modelo My First Statechart**.

A continuación, el editor le propone un **recorrido** (tour), que le sugerimos realizar a fin de familiarizarse con el mismo.

Recorridos (tour):

(?) Help -> Guides -> 1. **CREATE Web - User Interface**

(?) Help -> Guides -> 2. **CREATE Web – Shortcuts**



Documentos tener en cuenta:

[Five-minutes tutorial](#)

[Comprehensive tutorial](#)

[Statechart concepts tutorial](#)

[Statechart language reference](#)

Tarea para el hogar

Paso 03: Simular el [modelo](#) (Start Simulation).

Paso 04: Generar la [imagen](#) y exportar el [modelo](#) (Export Statechart).

En su PC **crear** una carpeta [my_first_statechart](#), **mover** el archivo exportado (.scm y .png) de la carpeta **descargas** a allí.

Paso 05: Almacenar la carpeta de la actividad, en el [repositorio](#) creado en [GitHub](#), en el **TP1 – Actividad 00 – Paso 01**.

TP1 – Actividad 02 – Sensor Statechart

Objetivos

Usar **Itemis Create** Cloud Editor

~~Usar **Itemis Create** Desktop~~

Generar proyecto **Itemis Create**

Editar/simular/depurar **diagrama de estado** (modelo): **Sensor Statechart** (Modeling)

Usar **Git** y **GitHub**

Generar/Actualizar repositorio por línea de **comando**

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Paso 01: Ejecutar **Itemis Create** Cloud Editor - ~~**Itemis Create** Desktop~~.

Paso 02: Generar un **modelo** nuevo, nombre: **sensor_statechart**.

Paso 03: Editar y simular el **modelo**, volcando allí el **comportamiento sensor** (un solo botón), analizado en el punto **TP1 – Actividad 00 – Paso 06 & TP1 – Actividad 00 – Paso 07**.

~~**Paso 04:** Generar la **imagen** del **modelo** (**File > Save As Image File ...**).~~

En su PC ~~crear~~ una carpeta **sensor_statechart**, ~~copiar~~ allí los archivos generados (**.ysc** y **.png**).

Paso 04: Generar la **imagen** y exportar el **modelo** (**Export Statechart**).

En su PC **crear** una carpeta **sensor_statechart**, **mover** los archivos exportados (**.scm** y **.png**) de la carpeta **descargas** a allí.

Paso 05: Almacenar la carpeta generada en la actividad **sensor_statechart**, en el **repositorio** creado en **GitHub**, en el **TP1 – Actividad 00 – Paso 01**.

TP1 – Actividad 03 – System Statechart

Objetivos

Usar **Itemis Create** Cloud Editor

~~Usar **Itemis Create** Desktop~~

Generar proyecto **Itemis Create**

Editar/simular/depurar **diagrama de estado** (modelo): **System Statechart** (Modeling)

Usar **Git** y **GitHub**

Generar/Actualizar repositorio por línea de **comando**

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Paso 01: Ejecutar **Itemis Create** Cloud Editor - ~~**Itemis Create** Desktop~~.

Paso 02: Generar un **modelo** nuevo, nombre: **system_statechart**.

Paso 03: Editar y simular el **modelo**, volcando allí el **comportamiento system** (un solo sistema), analizado en el punto TP1 – Actividad 00 – Paso 08 & TP1 – Actividad 00 – Paso 09.

~~**Paso 04:** Generar la **imagen** del **modelo** (File > Save As Image File ...).~~

En su PC ~~crear~~ una carpeta **system_statechart**, ~~copiar~~ allí los archivos generados (**.ysec** y **.png**).

Paso 04: Generar la **imagen** y exportar el **modelo** (Export Statechart).

En su PC **crear** una carpeta **system_statechart**, **mover** los archivos exportados (**.scm** y **.png**) de la carpeta **descargas** a allí.

Paso 05: Almacenar la carpeta generada en la actividad **system_statechart**, en el **repositorio** creado en **GitHub**, en el **TP1 – Actividad 00 – Paso 01**.

TP1 – Actividad 04 – Actuator Statechart

Objetivos

Usar **Itemis Create Cloud Editor**

~~Usar **Itemis Create Desktop**~~

Generar proyecto **Itemis Create**

Editar/simular/depurar **diagrama de estado** (modelo): **Actuator Statecharts** (Modeling)

Usar **Git** y **GitHub**

Generar/Actualizar repositorio por línea de **comando**

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Paso 01: Ejecutar **Itemis Create Cloud Editor** - ~~**Itemis Create Desktop**~~.

Paso 02: Generar un **modelo** nuevo, nombre: **actuator_statechart**.

Paso 03: Editar y simular el **modelo**, volcando allí el **comportamiento actuator** (un solo led), analizado en el punto **TP1 – Actividad 00 – Paso 10 & TP1 – Actividad 00 – Paso 11**.

~~**Paso 04:** Generar la **imagen** del **modelo** (**File > Save As Image File ...**).~~

~~En su PC **crear** una carpeta **actuator_statechart**, **copiar** allí los archivos generados (**.yxc** y **.png**).~~

Paso 04: Generar la **imagen** y exportar el **modelo** (**Export Statechart**).

En su PC **crear** una carpeta **actuator_statechart**, **mover** los archivos exportados (**.scm** y **.png**) de la carpeta **descargas** a allí.

Paso 05: Almacenar la carpeta generada en la actividad **actuator_statechart**, en el **repositorio** creado en **GitHub**, en el **TP1 – Actividad 00 – Paso 01**.

GitHub: Presionar: **Commit changes ...** (**Guardar** los cambios)

Tildar: **Create a new branch** for this commit and start a pull request

Presionar: **Propose changes**

Presionar: **Create pull request**, tipear el nombre del **branch** y **agregar** a los docentes auxiliares de su curso, como **Reviewers** (para invitarlos a **revisar** sus **entregas** mediante un **pull request**)

Ver **cómo hacer el pull request** en este video: [Trabajo colaborativo con Github](#)

En la descripción del video se detalla el contenido (ver en [14:23](#) la sección "**CÓMO SOLICITAR REVISIONES**")

IMPORTANTE: Se recomienda que primero vean "[19:25 Solución a un problema muy habitual](#)" y luego "[19:02 Create pull request y Review requested](#)"